Bluetooth Low Energy

Advertising, GATT, permissions, and a peripheral written in Go

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Read the link

Advertising, scanning, GAP roles, connection parameters and MTU, and what each costs in energy.

Model with GATT

Services, characteristics, descriptors and UUIDs, and when to read, write, notify or indicate.

Ask for the radio

Android 12 and later runtime permissions, the iOS usage string, and the background limits on both.

Build both sides

A Flutter central with flutter_blue_plus and a Go peripheral with one notifying characteristic.

PART 1 · THE LINK

Two radios, one name

Advertising, scanning, connecting, and the parameters that cost power

Classic Bluetooth and BLE are different radios

	CLASSIC (BR/EDR)	BLUETOOTH LOW ENERGY
Designed for	continuous streams: audio, files	short messages, seconds to hours apart
Connection	held open, always costing power	set up, used and torn down in milliseconds
Throughput	about a megabit per second	tens of kilobits per second
Idle current	milliamps	microamps between advertisements
Typical device	headset, speaker, keyboard	sensor, tag, wearable, beacon

They share a name, a band and a chip, and almost no protocol. `flutter_blue_plus` is BLE only: it will never see a Classic serial module.

Why BLE fits a sensor

The radio is off

A BLE device sleeps almost all the time. It wakes, sends a few bytes, and goes back down.

Short exchanges

An advertisement is under a millisecond of air time. A connection event can be as short.

Coin-cell budgets

A tag reporting once a second runs for months on a button cell. Wi-Fi cannot.

Forty channels

2.4 GHz in 2 MHz steps: three channels for advertising, thirty-seven for data.

Every phone has it

No access point, no IP address, no infrastructure. The phone is the gateway.

Not for bulk

Firmware images and audio over BLE are slow. Use it for readings and commands.

GAP: four roles, two that matter

Peripheral
Advertises, waits, accepts one connection. Your sensor. It owns the data and the GATT server.

Broadcaster
Advertises and never accepts a connection. A beacon. The data is in the advertisement itself.

Central
Scans, picks a peripheral, opens the connection, and drives every exchange. Your phone.

Observer
Scans and never connects. Useful when many tags report the same tiny value.

The central controls the timing. The peripheral proposes parameters and the central decides. Every power question is really a question about that decision.

GAP

Advertising: the peripheral talks first

What goes out

A short packet on the three advertising channels, repeated at an interval you pick, from about 20 ms to about 10 s.

How much fits

A legacy advertising packet carries 31 bytes. A scan response, requested by an active scanner, carries 31 more.

Connectable or not

A connectable advertisement invites a central. A non-connectable one is a broadcast: the payload is the message.

The interval is the bill

Advertising faster is found faster and costs more. This one knob decides a tag's battery life.

GAP

Scanning: passive, active, and duty cycle

Passive scan
Listen only. You see the advertisement and nothing more.

Active scan
The scanner sends a scan request and the peripheral replies with a scan response. More data, more air time, more current on both sides.

⚠ Scanning is expensive

Continuous scanning keeps the phone's receiver powered. It is the most common reason a BLE app drains a battery.

Filter in the platform, not in Dart. Passing a service UUID to the scan call pushes the filter into the operating system, so unwanted advertisements never cross the platform channel.

Connection parameters decide the power bill

PARAMETER	RANGE	WHAT IT DOES
Connection interval	7.5 ms to 4 s	how often the two radios meet
Peripheral latency	0 to 499 events	how many meetings the peripheral may skip
Supervision timeout	100 ms to 32 s	how long silence lasts before the link is lost

Latency is the free win. It keeps the link alive and the response fast when it matters, while letting the device sleep through the events where it has nothing to send.

MTU: how much fits in one packet

23 B

default ATT MTU, negotiated at connection time

20 B

payload left for one notification after the header

512 B

largest value a characteristic may hold

Android asks for a large MTU during connection and `flutter_blue_plus` does it for you. iOS negotiates on its own and gives you no control. A value larger than the MTU is fetched in several round trips.

Design to the default, not to the best case. Assume twenty usable bytes per notification until you have measured the negotiated MTU on the actual phone.

PART 2 · GATT

Data as a tree of attributes

Services, characteristics, descriptors, UUIDs, and the four operations

GATT is a tree of attributes

Service
A named group of related values. A device exposes several: yours, plus battery and device info.

Characteristic
One value, with properties that say what may be done to it, and a handle.

Descriptor
Metadata on a characteristic: a name, a unit, a range, or the subscription switch.

The one descriptor you will meet

The Client Characteristic Configuration Descriptor, `0x2902`, is how a central turns notifications on. Writing it is what `setNotifyValue` does.

The peripheral is a small database. The central discovers the schema after every connection.

UUIDs: 16 bits or 128

Adopted, 16-bit

Numbers assigned by the Bluetooth SIG for standard services: `0x180D` heart rate, `0x180F` battery.

Custom, 128-bit

A random UUID you generate once for your own service. Nobody assigns it and nobody will collide with it.

The short form expands

A 16-bit UUID is shorthand for a full one built on a fixed base. Only the short form travels, which saves bytes.

Prefer the standard one

If a standard service matches your data, use it. Test tools and the operating system already understand it.

Read, write, notify, indicate

OPERATION	STARTED BY	CONFIRMED	USE IT FOR
Read	central	yes	a value read once, on demand
Write	central	yes	a command that must not be lost
Write without response	central	no	commands where a loss is fine
Notify	peripheral	no	a value that changes often
Indicate	peripheral	yes	a rare event that must arrive

Notify for readings, indicate for events. A dropped temperature sample is replaced a second later. A dropped alarm is gone.

A heart-rate sensor, drawn out

LEVEL	UUID	PROPERTIES	MEANING
Service	0x180D		Heart Rate
Characteristic	0x2A37	notify	measurement: flags, then the rate
Descriptor	0x2902	read, write	the subscription switch
Characteristic	0x2A38	read	body sensor location, one byte
Characteristic	0x2A39	write	control point: reset the counter

Copy this shape: one characteristic for the reading, one for metadata, one for control.

PART 3 · SECURITY

Pairing, bonding, and what is encrypted

Association models, security levels, and private addresses

SECURITY

Pairing, bonding, and what they buy

Pairing

The two devices agree on keys and turn on link-layer encryption. It lasts until the link drops.

Bonding

Pairing, then storing the long-term key on both sides. The next connection encrypts with no user interaction.

Neither is automatic

A plain BLE link is unencrypted. Encryption starts only when a characteristic demands it.

Bond once, then forget about it. A bonded sensor reconnects silently. An unbonded one asks the user every time, and users say no.

Association models and security levels

MODEL	NEEDS	PROTECTS AGAINST
Just Works	nothing	a passive listener, and nothing else
Passkey Entry	a display on one side, a keypad on the other	a device in the middle, if the user reads it
Numeric Comparison	a display and a yes/no button on both sides	the same, with less typing
Out of Band	a second channel, such as an NFC tap	the same, with nothing to compare

No display means no protection from a device in the middle. A screenless sensor can only pair with Just Works. If the data matters, sign the payload above BLE.

SECURITY

Addresses, privacy, and what is encrypted

Advertisements are in the clear
Anyone with a radio reads your name, your service UUID and any broadcast payload. Put nothing personal there.

Encryption covers the link
After pairing, reads, writes and notifications are all encrypted.

Addresses rotate
A private device advertises an address that changes periodically. Only a bonded peer can resolve it.

⚠ Do not key on the address

Both platforms hand you an identifier that is stable for your app and useless anywhere else. On iOS it is not the hardware address at all.

PART 4 · THE PHONE

Asking for the radio

Android and iOS permissions, and flutter_blue_plus end to end

Permissions: two steps, both mandatory

Step one is static
You declare at build time what the app may ever ask for. Android reads the manifest, iOS reads Info.plist and the sentence shown to the user.

Step two is at runtime
The user sees a system dialog you cannot style, move or reword, and may refuse.

Order of operations

Declare, then check the adapter state, then request, then scan. Scanning first gives you an unauthorized adapter and no useful error.

Permissions are a build-config problem first. An undeclared Android permission is denied forever with no dialog. A missing iOS usage string terminates the app on first use.

Android: the manifest entries

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-feature android:name="android.hardware.bluetooth_le"
              android:required="true"/>

<!-- Android 12 and later: runtime permissions, no location -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
                 android:usesPermissionFlags="neverForLocation"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>

<!-- Android 11 and earlier: install-time, plus location to scan -->
<uses-permission android:name="android.permission.BLUETOOTH"
                 android:maxSdkVersion="30"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
                 android:maxSdkVersion="30"/>
```

Android: neverForLocation and the scan story

VERSION	TO SCAN	TO CONNECT
Android 12 and later	BLUETOOTH_SCAN at runtime	BLUETOOTH_CONNECT at runtime
Android 10 and 11	ACCESS_FINE_LOCATION, location switched on	BLUETOOTH, at install
Android 9 and older	ACCESS_COARSE_LOCATION at runtime	BLUETOOTH, at install

Do not ask for location to read a sensor. The `neverForLocation` flag promises the system you do not derive position from what you scan.

PERMISSIONS

iOS: Info.plist and background modes

```
<!-- ios/Runner/Info.plist -->
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Shows the live reading from your lab sensor.</string>

<!-- Only when the app must keep working with the screen off -->
<key>UIBackgroundModes</key>
<array>
  <string>bluetooth-central</string>
</array>
```

Background BLE is narrow on both platforms. iOS wakes the app briefly for a subscribed value and refuses an unfiltered background scan. Android needs a foreground service.

flutter_blue_plus: scanning

```
await FlutterBluePlus.adapterState
    .where((s) => s == BluetoothAdapterState.on).first;

final sub = FlutterBluePlus.onScanResults.listen(
    (results) => emit(results), onError: (e) => log('$e'));
FlutterBluePlus.cancelWhenScanComplete(sub);

await FlutterBluePlus.startScan(
    withServices: [Guid('180D')], timeout: Duration(seconds: 15));
```

Always pass a filter and a timeout. `onScanResults` clears between scans, `scanResults` keeps the previous batch, and `cancelWhenScanComplete` removes the listener for you.

Connect, then discover

```
final sub = device.connectionState.listen((s) {
  if (s == BluetoothConnectionState.disconnected) {
    log('${device.disconnectReason?.code}');
  }
});
device.cancelWhenDisconnected(sub, delayed: true, next: true);

await device.connect();
final services = await device.discoverServices();
```

Rediscover after every reconnection. Service and characteristic objects belong to one connection. Reusing them after a drop throws, and that is the most common exception in this lab.

Subscribe to a characteristic

```
final c = services
    .firstWhere((s) => s.uuid == Guid('180D'))
    .characteristics
    .firstWhere((c) => c.uuid == Guid('2A37'));

final sub = c.onValueReceived.listen((v) => emit(decode(v)));
device.cancelWhenDisconnected(sub);

await c.setNotifyValue(true);   // writes the CCCD for you
```

Listen first, then enable. Enabling notifications before the listener exists loses whatever the peripheral sends in between.

Disconnect, and reconnect on purpose

```dart
await device.disconnect();

// Later, with no scan: the id was saved on the first connect.
final d = BluetoothDevice.fromId(savedRemoteId);
await d.connect(autoConnect: true, mtu: null);
await d.connectionState
    .where((s) => s == BluetoothConnectionState.connected).first;
await d.discoverServices();
```

A reconnect loop is part of the feature. Sensors go out of range, run flat and get switched off. An app that needs a manual retry after every drop is not finished.

PART 5 · THE PERIPHERAL

The other side, written in Go

tinygo.org/x/bluetooth, the two lab set-ups, and how to size a characteristic

What tinygo.org/x/bluetooth supports

HOST	UNDERNEATH	CENTRAL	PERIPHERAL
Linux, Windows	BlueZ, WinRT	yes	yes
macOS	CoreBluetooth	yes	no, central only
Nordic nRF52840	SoftDevice	yes	yes
ESP32-C3, ESP32-S3	espradio, over HCI	yes	yes
Original ESP32	HCI co-processor	with firmware	with firmware

macOS cannot advertise. A Mac can be the central for a test, never the peripheral. Run the Go peripheral on Linux, on Windows, or on a board.

A peripheral in Go

```
var adapter = bluetooth.DefaultAdapter
var value bluetooth.Characteristic

func main() {
    must(adapter.Enable())
    adv := adapter.DefaultAdvertisement()
    must(adv.Configure(bluetooth.AdvertisementOptions{
        LocalName:    "MEC Lab",
        ServiceUUIDs: []bluetooth.UUID{serviceUUID},
    }))
    must(adv.Start())
    addService()
}
```

One service, one notifying characteristic

```
must(adapter.AddService(&bluetooth.Service{
    UUID: serviceUUID,
    Characteristics: []bluetooth.CharacteristicConfig{{
        Handle: &value,
        UUID:   charUUID,
        Flags:  bluetooth.CharacteristicNotifyPermission,
    }},
}))
// Writing the handle pushes a notification to every subscriber.
for i := uint8(0); ; i++ {
    value.Write([]byte{i})
    time.Sleep(time.Second)
}
```

Two set-ups for Lab 3

A board
An nRF52840 development board, flashed with `tinygo flash`. Runs on its own power and behaves like a real sensor.

An ESP32-C3 or ESP32-S3 also works, through espradio. The original ESP32 does not.

A laptop
The same program under `go run`, on Linux or Windows. No hardware, no flashing, and a peripheral you can restart.

macOS is central only. Use a Linux virtual machine with the adapter passed through, or a board.

Both set-ups are graded the same. The Flutter side is identical either way. Pick whichever reaches a working notification fastest, and pick it before the lab.

DESIGN

Design the service, not just the code

Keep the value small

Fit one reading in twenty bytes. Scaled integers beat a JSON string every time.

Batch on the device

Ten readings in one notification cost one connection event. Ten notifications cost ten.

Summarize, do not stream

Send a minimum, a maximum and a mean instead of a hundred raw samples.

One concern each

A reading, a setting and a command are three characteristics, not one blob.

Document the encoding

Byte order, scale and unit go in the repository next to the UUID.

Let the device sleep

A long interval with peripheral latency saves more than any code you write.

RECAP

Three things to remember

1. **The central owns the timing.** Connection interval, peripheral latency and MTU decide latency and battery life.
2. **GATT is discovered, never assumed.** Rediscover services after every reconnection and never reuse an old characteristic object.
3. **Declare before you request.** BLUETOOTH_SCAN with neverForLocation and BLUETOOTH_CONNECT on Android, NSBluetoothAlwaysUsageDescription on iOS.

FURTHER READING

pub.dev/packages/flutter_blue_plus · github.com/tinygo-org/bluetooth · Android Bluetooth permissions

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel